New Module Assignment

Student Guide — Fusion-Integrated

How a student picks up a new-module assignment and runs `new-module-prompt.md` end to end. This is the wrapper around that prompt, not a replacement for it — read both.

This is real work on a real platform. The module you are assigned will go into production if it clears review. There is no simplified version of the standard for a student assignment — §8 of the prompt applies exactly as written. It already includes what still applies from `definition-of-done.md`; where the two differ, the prompt governs.

You have to follow these steps in order. Do not skip ahead to Step 5 because you already know what the prompt says — Steps 1 through 4 are what make the prompt work for your specific module, and every one of them is something you do, not something that happens for you.

Step 1 — Confirm your assignment

You already have these three things, given to you along with this guide. Before you open anything else, you have to make sure you actually have all three written down — do not start Step 2 on a half-remembered version of any of them:

1. **Module name** — what you are building.
2. **Spec folder path** — where the BR (business rules), UC (use cases), and workflow documents for your module live. If this does not exist yet, you have to say so now — writing a module against a spec you wrote yourself is not this exercise.
3. **Module code** — a short, lowercase, singular-ish slug for it (no underscores), already agreed for your module. Do not change it on your own.

Do not guess any of these. Getting one wrong — especially the environment value in Step 4 — sends you reading for files that do not exist on your machine.

Step 2 — Read your own module's requirement specification documents

`modules/placement` is the reference module the **prompt** already tells the AI to read, as part of its own Phase 0 (Step 1.2) — that reading is the AI's job, not yours, and you do not need to open that code yourself.

Your job here is different: you have to read every BR (business rule), UC (use case), and workflow document in your `SPEC_FOLDER_PATH` yourself, before you ever open the prompt. You need to actually understand what you are asking the AI to build, because:

- Phase 1 (Step 2.1–2.8 of the prompt) asks **you** real questions — you have to answer them from knowledge of your own spec, not guess at what sounds right.
- Phase 2's blueprint needs **your** approval before any code gets written — approving something you have not read defeats the entire point of that gate.
- If the AI derives a role in Step 2.4 because nobody handed it a role list, you are the one who has to catch it if that derived role is wrong — you cannot catch a wrong reading of your spec if you never read the spec yourself.

Step 3 — Set up your environment

You get the **module-only** environment — `Fusion_System_Administrator` and `Fusion-Integrated` checked out, `fusionlab` restored from the public `fusion-dev.dump` (Fusion-README). You will not be given `Fusion` or `Fusion-client` checkouts, and you do not need them — the prompt's §1 tells you exactly what to read instead.

You have to confirm all of this before you go further:

- [ ] `docker compose up -d` brings up Postgres and both Redis instances, healthy; the IAM (from `Fusion_System_Administrator`) and the platform (`make dev`) are running
- [ ] `cd client && npm run dev` runs standalone — this is where you will demo your module
- [ ] You can log in with a test account and see `modules/placement` working, so you know the fixture is actually loaded

Step 4 — Fill in the prompt's six blanks, before you paste anything

Open `new-module-prompt.md` but do not paste it yet. You have to write down the six values that go in it first, using what you got in Step 1:

Blank	Your value	Where it came from
`MODULE_NAME`	you fill this in	already given to you — Step 1
`SPEC_FOLDER_PATH`	you fill this in	already given to you — Step 1
`ID_PREFIX`	you fill this in	you read it off the spec documents yourself, once you have the folder
`module_code`	you fill this in	already given to you — Step 1
`ENVIRONMENT`	module-only	fixed — this is the value for every student assignment
`NN`	you fill this in	the number of your module's docs/ folder, from your mentor

You have to fill these into the prompt text itself, everywhere they appear, before you paste it. Do not paste the prompt with the bracketed placeholders still in it and fill them in afterward — the AI session needs the real values from the first line.

Step 5 — Paste the prompt into a fresh AI session

You have to copy everything between `--- PROMPT BEGINS ---` and `--- PROMPT ENDS ---`, with your six blanks already filled in, into a **brand-new** AI coding session — not one that already has other context in it. Do not summarise the prompt, do not trim it, do not paraphrase a section you think you already understand. The length is deliberate.

Step 6 — Work through the prompt's own steps

From here the prompt drives itself — it is itself written as an explicit numbered sequence (Step 0.1 through Step 10.4, covering read → questions → blueprint → backend → tests → frontend → ops). At each one, you have to **answer what it asks and get approval where it asks for approval** — you do not get to skip ahead because a later step looks easy.

Two stops matter more than the rest, because they are where a student assignment most often goes wrong:

- **You must not write any application code before your mentor approves the Step 3.1–3.8 blueprint.** You have to post the blueprint (`docs/[NN]-[module_code]/[module_code]-blueprint.md`) and wait for a yes. The blueprint is the review, not a formality before the real work.
- **You have to stop and report, not guess, when**: a spec file is missing, two spec documents contradict each other, the spec conflicts with the platform's architecture, or you would otherwise have to invent the intended behaviour — including a role nobody gave you and no BR/UC/workflow document names (Step 2.4 of the prompt). Report the specific gap. An assignment where you report three honest open questions is worth more than one where you guessed and built on it.

The one thing you resolve yourself, no exception: if matching the spec or the reference modules would weaken security or cross a data-isolation boundary, you refuse it and say why — you do not escalate this one, you just refuse it.

Step 7 — Create your own branch

You have to fork `Fusion-Integrated` if you have not already, then create **your own branch** off `main` — you do not share a branch with another student, and you do not commit to `main` directly. Name it for your module: `module/<module_code>`.

Step 8 — Commit correctly

- You have to commit under **your own GitHub identity**. No AI attribution anywhere — not in code, comments, commit messages, or the PR body. This is §8 item 8 of the prompt, and it is not negotiable.
- You have to keep production data out of every commit, PR body, and doc — no real roll numbers, names, or person ids. Use the synthetic `fusion-dev.dump` accounts for every example instead.
- You must not commit, push, or open the PR until your mentor tells you the module is ready for it. An earlier “go ahead” does not carry forward to the next phase — you have to ask again.

Step 9 — Open your pull request

You have to open the PR **from your branch to** `main` on `FusionIIIT/Fusion-Integrated` — not to any other base branch, and not to your fork's own `main`. Link the blueprint and the spec folder it implements in the description.

Step 10 — Self-check against the definition of done, before requesting review

You have to pull the full checklist from the prompt's §8, which already includes what still applies from `definition-of-done.md`, and run it on yourself before you ask anyone to review. The short version:

[ ] `make check` and `make check-client` both green — tsc finished, not just npm test

[ ] Every live spec id is implemented and cited at its enforcement point, or listed as a known gap with a reason; no retired id cited anywhere

[ ] Fresh database → migrate → seed → grant → first user action succeeds — you have to prove this by doing it once, not by reasoning about it

[ ] The adversarial pass (Step 5.11 of the prompt) was actually run, not skipped

- [ ] Every scheduled task was run manually once
- [ ] No comment longer than one line anywhere in the diff
- [ ] No credential anywhere in a tracked file
- [ ] The plain user — the one with no interesting roles — was tested, not only the interesting accounts
- [ ] Every role in your permission list traces back to Step 1's assignment or to a cited BR/UC/workflow id from Step 2.4 of the prompt — none of them invented

If you cannot check every box, you have to say which ones are open in the PR description. A PR that lists its own gaps honestly is reviewable; one that claims a green checklist it did not actually run is not.

If you get stuck

You have to re-read §0's stop conditions in the prompt before asking. If what you have is genuinely one of them — a missing file, a contradiction, an architecture conflict, or a guess you'd otherwise have to make — bring the specific thing, not a general “I'm stuck.” If it is not one of those, it is probably answered already in the prompt's Step 1.1 through Step 3.8; re-read the relevant step yourself before escalating.